

PROJECT SUBMISSION

ServiceNow Incident Form Configuration
UI Policy, UI Policy Action, Client Scripts & Testing

Project	ServiceNow Incident Configuration
Application	ServiceNow
Table	Incident [incident]
Configuration Areas	UI Policy, UI Policy Action, Client Scripts
Milestones	6

Project Objective: Configure the ServiceNow Incident form using a UI Policy, UI Policy Action, and client-side scripts to demonstrate dynamic field behavior, save-time validation, and list editing validation.

Milestone 1: Create UI Policy on Incident

Activity 1: Create UI Policy

Navigate to **All > System UI > UI Policies** and create a new UI Policy for the Incident table.

Field	Configuration
Table	Incident [incident]
Active	True
Short description	Urgency-based Incident UI Policy
Condition	Define the required condition for the project (for example, Priority/Urgency condition).
Reverse if false	Enable when the form should return to the default state when the condition becomes false.

Purpose: A UI Policy controls the behavior of fields on the Incident form without requiring server-side processing. It can make fields mandatory, visible, or read-only based on a condition.

Milestone 2: Create UI Policy Action – Urgency

Create a UI Policy Action associated with the UI Policy from Milestone 1. Select the **Urgency** field and configure its behavior according to the project requirement.

Setting	Value
Field	Urgency
Mandatory	Configure as required by the condition
Visible	True
Read-only	Configure as required by the condition

Expected Result: When the UI Policy condition is met, the Urgency field changes to the configured state.

Milestone 3: Create onChange Client Script

Create an **onChange Client Script** on the Incident table. The script runs whenever the selected field value changes and can immediately update the form.

Recommended structure:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue == '') {
        return;
    }

    // Add the project-specific condition here.
    // Example:
    // if (newValue == '1') {
    //     g_form.setMandatory('urgency', true);
    // } else {
    //     g_form.setMandatory('urgency', false);
    // }
}
```

Purpose: Provides immediate client-side feedback when a field changes, improving the user experience before the record is submitted.

Milestone 4: Create onSubmit Client Script (Save Validation)

Create an **onSubmit Client Script** to validate the Incident before it is saved. Return **false** when validation fails and **true** when the record can be submitted.

```
function onSubmit() {
    // Add the project-specific validation condition here.
    // Example:
    // if (g_form.getValue('urgency') == '') {
    //     g_form.addErrorMessage('Urgency must be selected before saving.');
```

```
    // return false;
    // }

    return true;
}
```

Purpose: Prevents invalid Incident records from being submitted and provides a clear message to the user.

Milestone 5: Create onCellEdit Client Script

Create an **onCellEdit Client Script** for list editing. This script validates or controls changes made directly from an Incident list rather than from the full form.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    var save = true;

    // Add the project-specific validation here.
    // Example:
    // if (newValue == '') {
    //     save = false;
    // }

    callback(save);
}
```

Purpose: Ensures that users cannot bypass the intended validation by editing Incident records directly from a list.

Milestone 6: Testing the Configuration

Test Case	Action	Expected Result
1. UI Policy	Open an Incident and meet the policy condition.	Urgency changes to the configured state.
2. UI Policy Reverse	Change the condition so it is false.	Urgency returns to the default state if Reverse if false is enabled.
3. onChange	Change the field that triggers the client script.	Configured field behavior updates immediately.
4. onSubmit – valid	Enter valid values and save.	Incident saves successfully.
5. onSubmit – invalid	Leave an invalid combination/value and save.	Save is blocked and an error message appears.
6. onCellEdit	Edit the relevant field from the Incident list.	Invalid list edit is rejected; valid edit is accepted.

Testing Checklist: Verify each script is Active, confirm the correct Table and Field are selected, test both valid and invalid values, and confirm that browser console errors are not generated.

Conclusion

The project demonstrates how ServiceNow client-side configuration can improve Incident form behavior and data quality. The UI Policy and UI Policy Action provide dynamic form control, while the onChange, onSubmit, and onCellEdit Client Scripts provide immediate behavior, save-time validation, and list-edit validation. Together, these configurations create a consistent and controlled Incident management experience.

Submission Note: Replace the example conditions in the script sections with the exact conditions specified by your assignment before submitting the project.